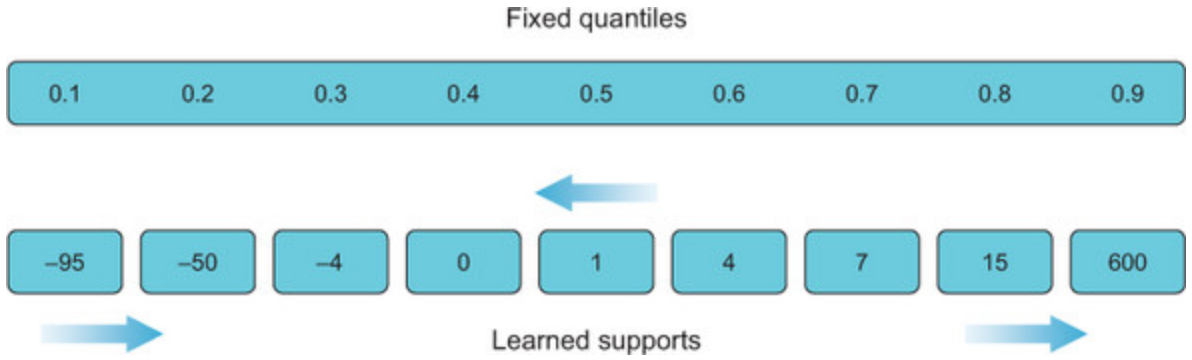
Dist-DQN learned to predict three full probability distributions rather than a single action value, so these auxiliary tasks force the algorithm to learn more robust abstractions.

We also discussed a significant limitation in the way we've implemented Dist-DQN, namely that we're using discrete probability distributions with finite support, so we can only represent action values within a very small range, from –10 to 10. We could make this range wider at the cost of more computational processing, but we can never represent an arbitrarily small or large value with this approach. The way we've implemented it is to use a fixed set of supports but learn the set of associated probabilities.

One fix to this problem is to instead use a fixed set of probabilities over a variable (learned) set of supports. For example, we can fix our probability tensor to range from 0.1 to 0.9, e.g., `array([0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])`, and we instead have our Dist-DQN predict the set of associated supports for these fixed probabilities. That is, we're asking our Dist-DQN to learn what support value has a probability of 0.1, and 0.2, and so on. This is called *quantile regression* because these fixed probabilities end up representing quantiles of the distribution (figure 7.23). We learn the supports at and below the 50th percentile (probability 0.5), the 60th percentile, and so on.

Figure 7.23. In quantile regression, rather than learning what probabilities are assigned to a fixed set of supports, we learn a set of supports that correspond to a fixed set of probabilities (quantiles). Here you can see that the median value is 1 since it is at the 50th percentile.



With this approach, we still have a discrete probability distribution, but we can now represent any possible action value—it can be arbitrarily small or